

Dokumentation

Programmmentwurf ASE TINF23BI

31.05.2026

Maximilian Schelkle

Inhalt

1. Domain-Driven Design	I
1.1. Analyse der Ubiquitous Language	I
1.2. Verwendung taktischer Muster des DDD	3
1.3. Analyse und Begründung der verwendeten Muster	3
1.3.1. Value Object: Email	3
1.3.2. Entity: ConsumptionProgress	4
1.3.3. Aggregate: Account	4
1.3.4. Repository: AccountRepository	5
1.3.5. Domain Service: AccountService	5
2. Clean Architecture	6
2.1. Schichtarchitektur planen und begründen	6
2.2. Umsetzung von zwei Schichten	6
3. Programming Principles	6
3.1. Analyse und Begründung	6
3.1.1. Single Responsibility Principle (SRP)	6
3.1.2. Open/Closed Principle (OCP)	7
3.1.3. Liskov Substitution Principle (LSP)	7
3.1.4. Interface Segregation Principle (ISP)	7
3.1.5. Dependency Inversion Principle (DIP)	7
3.2. Aufgabe 4: Unit Tests	8
3.2.1. Unit Tests und ATRIP-Regeln	8
3.2.2. Einsatz von Mocks	8
4. Refactoring	9
4.1. Code Smells	9
4.1.1. Inconsistent Exception Handling	9
4.1.2. Duplicate Code	9
4.1.3. Large Class	9
4.1.4. Speculative Generality	9
4.1.5. Switch Statements (Type Code)	10
4.2. Refactorings	10
4.2.1. Extract Method	10
4.2.2. Replace Conditional with Polymorphism / Replace Type Code	10
5. Entwurfsmuster	10
5.1. Einsatz eines Entwurfsmusters: Builder	10

I. Domain-Driven Design

I.1. Analyse der Ubiquitous Language

Die folgende Tabelle dokumentiert die Ubiquitous Language des Projekts.

Term	Definition / Aufgaben	Regeln
Medialtem	Das abstrakte Konzept jedes verwalteten Medientypes im System (z.B. Buch, Film, Serie,...). Medialtem sind die zu verwaltenden Objekte des Systems und definieren die Attribute des jeweiligen Items.	• Medialtems haben einen Besitzer (Owner Account) und können nur von diesem geändert werden. • Medialtems können nicht gelöscht werden, da andere Accounts von ihnen abhängen. • Medialtems sind global von allen Accounts aus nutzbar. • Medialtems definieren, wie ihr Konsumstatus berechnet wird (Seiten, Zeit, etc.)
Review	• Jeder Nutzer kann für ein Medialtem ein Review anlegen. • Es besteht aus einem Rating und einem optionalen Kommentar. • Alle Änderungen an dem Review werden in einer Historie gespeichert. • Zudem speichert jedes Review den Konsumfortschritt des Medialtems von dem Account. • Reviews sind global von allen Accounts aus einsehbar.	Ein Review kann nur nach Konsumbeginn eines Medialtems angelegt werden, d.h. nicht konsumierte Medialtems können kein Review des Accounts haben.
Rating	Ein Rating ist ein Wert im Intervall [0.0, 10.0] und Teil eines Reviews.	• Ratings werden auf das Intervall [0.0, 10.0] normalisiert. • Das Rating für Medialtems mit Untergeordneten Medialtems (z.B. Serien -> Staffeln) ergibt sich aus dieser Formel: $\frac{\sum \text{EpisodenRatings} + \sum \text{DirekteRatings}}{ \text{EpisodenRatings} + \text{DirekteRatings} }$
BookShelf	Das BookShelf stellt die zentrale Bibliothek eines Nutzers dar, welche dieser verwalteten kann.	Das BookShelf verwaltet Playlists, Watchlists und die Einträge (BookShelfEntry) von Medialtems.

BookShelfEntry	Jedes Medialtem, dass von einem Account konsumiert/genutzt wird und deshalb in dessen BookShelf gespeichert ist. Es speichert den Konsumfortschritt des Accounts an dem Medialtem.	• Jedes Medialtem kann genau einmal als BookShelfEntry in dem BookShelf eines Accounts vorhanden sein. • Der Konsumstatus eines BookShelfEntry startet im Zustand <i>NOT_STARTED</i> und geht über in <i>STARTED</i> oder <i>COMPLETED</i> je nachdem, wie viel der Account das Medialtem konsumiert hat. • Der Konsumstatus kann nie wieder in <i>NOT_STARTED</i> übergehen, nachdem dieser Zustand verlassen wurde, auch wenn das Medialtem von Beginn an rekonsumiert wird.
Label	Jeder BookShelfEntry kann von einem Account mit einem eindeutigen Set an Label's kategorisiert werden.	• Ein Label kann nur einmal für denselben BookShelfEntry genutzt werden. • Ein Label besteht aus Kleinbuchstaben und Leerzeichen.
Playlist	Playlists sind geordnete Sammlungen an BookShelfEntry's und können von einem Account für dessen BookShelf erstellt werden.	• Jedes Medialtem in einer Playlist muss ein BookShelfEntry sein. • Die Reihenfolge ist zunächst InsertionOrder, kann aber vom Account geändert werden. • BookShelfEntry's können mehrmals in einer Playlist vorhanden sein.
Watchlists	Watchlists sind ungeordnete Sammlungen an einzigartigen BookShelfEntry's und können von einem Account für dessen BookShelf erstellt werden.	• Jedes Medialtem in einer Playlist muss ein BookShelfEntry sein. • BookShelfEntry's können jeden Konsumstatus haben und in einer Watchlist sein. • Jeder BookShelfEntry darf nur einmal in einer Watchlist vorhanden sein.
Account	Ein Account stellt einen Nutzer des Systems dar. Accounts werden genutzt, um Nutzer zu authentifizieren/autorisiern und indentifizieren.	• Jeder Nutzer ist über seinen Account eindeutig identifizierbar.

		• Löschen eines Accounts macht diesen nicht mehr authentifizierbar. • Account Nutzernamen sind eindeutig, 2 bis 30 Zeichen lang und können aus Nummern, Buchstaben und Unterstrichen bestehen.
--	--	---

I.2. Verwendung taktischer Muster des DDD

Im Projekt werden die folgenden taktischen Muster des DDD verwendet:

- **Value Objects:** Zur Repräsentation von Konzepten, die durch ihre Attribute definiert sind und keine eigene Identität haben.
- **Entities:** Zur Repräsentation von Objekten mit einer eindeutigen Identität, die über die Zeit hinweg bestehen bleibt.
- **Aggregates:** Zur Gruppierung von Entities und Value Objects, die als eine Einheit behandelt werden.
- **Repositories:** Zur Abstraktion der Persistenzschicht und zum Zugriff auf Aggregate.
- **Domain Services:** Zur Kapselung von Geschäftslogik, die nicht in eine einzelne Entity oder ein Value Object passt.

I.3. Analyse und Begründung der verwendeten Muster

I.3.1. Value Object: Email

Die Klasse Email (`org.alcey.bookshelf_pro_domain.account.Email`) ist als Value Object implementiert.

```
@ValueObject
public record Email(@jakarta.validation.constraints.Email String email) {
    private static final EmailValidator validator = EmailValidator.getInstance();
    public Email {
        if (!validator.isValid(email)) {
            throw new IllegalArgumentException("Email " + email + " is not valid.");
        }
    }
}
```

Begründung: Eine E-Mail-Adresse hat keine eigene Identität, sondern wird durch ihren Wert definiert. Zwei Email-Objekte mit derselben Adresse sind austauschbar. Die Klasse ist ein Record, was sie und ihre Attribute `final` macht, was die Unveränderlichkeit sicherstellt. Die Validierung im Konstruktor stellt sicher, dass nur gültige E-Mail-Adressen erstellt werden können.

I.3.2. Entity: ConsumptionProgress

Die Klasse `ConsumptionProgress` (`org.alcey.bookshelf_pro_domain.bookshelf.bookshelf_entry.consumption.ConsumptionProgress`) ist ein Entity.

```
@Entity
public final class ConsumptionProgress {
    @Identity
    private final ConsumptionProgressId id;
    private ConsumptionState state;
    @Valid
    private MediaItemConsumptionProgress progress;
    public ConsumptionProgress(
        ConsumptionProgressId id,
        MediaItemConsumptionProgress progress,
        ConsumptionState initialState
    ) {
        this.id = id;
        this.state = initialState.nextState(progress);
        this.progress = progress;
    }
    public void updateProgress(MediaItemConsumptionProgress newProgress) {
        this.state = state.nextState(newProgress);
        this.progress = newProgress;
    }
    // ...
}
```

Begründung: Ein `ConsumptionProgress` hat eine eindeutige `ConsumptionProgressId` und einen Lebenszyklus (`state`, `progress`). Der Zustand eines `ConsumptionProgress` kann sich im Laufe der Zeit ändern, aber die Identität bleibt dieselbe. Dies ist wichtig, da Zustandsübergänge des `ConsumptionProgress` in der Domäne modelliert sind.

I.3.3. Aggregate: Account

Das Account-Aggregat (`org.alcey.bookshelf_pro_domain.account.Account`) fasst die Value Objects `Email`, `Username` und `Password` und die `deleted` Flag zusammen und ist identifizierbar durch `AccountId`.

```
@AggregateRoot
public final class Account {
    @Identity
    private final AccountId id;
    @Valid
    private Username name;
    @Nullable
    @Valid
    private Email email;
    @Valid
    private Password password;
    private boolean deleted;
```

```
// ...  
}
```

Begründung: Das Account-Aggregat stellt sicher, dass alle Änderungen an einem Account als eine atomare Operation behandelt werden. Es hat einen Zustand (der Account eines Nutzers ändert sich über die Zeit, aber bleibt derselbe Account).

I.3.4. Repository: AccountRepository

Das AccountRepository (org.alcey.bookshelf_pro_domain.account.AccountRepository) ist ein Repository für das Account-Aggregat.

```
@Repository  
public interface AccountRepository {  
    Optional<Account> findById(AccountId id);  
    Optional<Account> findByUsername(Username username);  
    void save(Account account);  
    void update(Account account);  
    void delete(AccountId id);  
    boolean existsUsername(Username name);  
}
```

Begründung: Das Repository abstrahiert die Persistenzlogik für Account-Aggregate. Es bietet Methoden zum Abrufen und Speichern von Account-Objekten, ohne dass der aufrufende Code die Details der Datenbankimplementierung kennen muss.

I.3.5. Domain Service: AccountService

Der AccountService (org.alcey.bookshelf_pro_domain.account.AccountService) ist ein Domain Service.

```
@Service  
public final class AccountService {  
    private final AccountRepository accountRepository;  
    public AccountService(AccountRepository accountRepository) {  
        this.accountRepository = accountRepository;  
    }  
    public Account createAccount(/* ... */) {  
        // ...  
    }  
    public void changeUsername(/* ... */) {  
        // ...  
    }  
    private void validateUserNameIsUnique(Username name) {  
        // ...  
    }  
}
```

Begründung: Die Erstellung und Änderung eines Accounts erfordert die Koordination mit dem AccountRepository, um die Bedingung eines eindeutigen Username sicherzustellen. Diese Logik passt nicht in die Account-Entity selbst, da sie externe Abhängigkeiten hat. Der AccountService

kapselt diese Logik und stellt sicher, dass ein Account nur in einem konsistenten Zustand erstellt und verändert wird.

2. Clean Architecture

2.1. Schichtarchitektur planen und begründen

Die Anwendung folgt einer klassischen Clean Architecture, die in drei Hauptschichten unterteilt ist:

- **Domain Layer:** Enthält die Kernlogik der Anwendung, einschließlich Entities, Value Objects, Aggregates, Repositories und Domain Services. Diese Schicht ist unabhängig von jeglicher Technologie oder Infrastruktur.
- **Application Layer:** Orchestriert die Anwendungsfälle (Use Cases) der Anwendung. Sie verwendet die Domain-Objekte, um die Geschäftsregeln auszuführen.
- **Plugin/Infrastructure Layer:** Implementiert die Details der Infrastruktur, wie z.B. die Datenbank, die REST-API und die Security.

Begründung: Diese Architektur entkoppelt die Geschäftslogik von der Infrastruktur, was die Wartbarkeit, Testbarkeit und Flexibilität der Anwendung erhöht. Die Abhängigkeiten zeigen immer nach innen, von den äußeren Schichten (Infrastruktur) zu den inneren Schichten (Domain).

2.2. Umsetzung von zwei Schichten

Im Projekt sind alle drei Schichten zu großen Teilen umgesetzt.

- Die **Domain Layer** befindet sich im Modul `bookshelf-pro-domain`. Es enthält alle Domänenobjekte wie `Account`, `Book`, `Review` etc. und deren Geschäftsregeln.
- Die **Application Layer** befindet sich im Modul `bookshelf-pro-application`. Es enthält die Use Cases wie `CreateAccountUseCase`, `AddReviewUseCase` etc., die die Domänenobjekte zur Ausführung der Anwendungslogik verwenden.
- Die **Plugin Layer** befindet sich im Modul `bookshelf-pro-plugins`. Es enthält die REST-Endpunkte wie `AccountController` etc., die Datenbankschnittstellen (z.B. `JooqMediaItemRepository`) und die Spring Security Logik.

3. Programming Principles

3.1. Analyse und Begründung

3.1.1. Single Responsibility Principle (SRP)

Beschreibung: Jede Klasse sollte nur eine einzige Verantwortung haben.

Beispiel: Die Klasse `AddBookshelfEntryUseCase` (`org.alcey.bookshelf_pro_application.bookshelf.bookshelf_entry.AddBookshelfEntryUseCase`) ist nur für die Orchestrierung des Anlegens eines neuen Eintrags im Bücherregal verantwortlich.

Begründung: Die Klasse nimmt einen `AddBookshelfEntryCommand` entgegen, validiert die Eingabe, erzeugt die notwendigen IDs, erstellt die Entität `BookshelfEntry` und speichert diese über ein Repository. Sie enthält selbst keine Geschäftslogik der Domain-Objekte. Dabei nutzt sie die Methoden und Implementierungen in anderen Klassen.

3.1.2. Open/Closed Principle (OCP)

Beschreibung: Software-Entitäten (Klassen, Module, Funktionen usw.) sollten offen für Erweiterungen, aber geschlossen für Modifikationen sein.

Beispiel: Das `MediaItem`-Interface (`org.alcey.bookshelf_pro_domain.media_item.MediaItem`) und seine Implementierungen (`Book`, `Movie`).

Begründung: Neue Medientypen können durch die Implementierung des `MediaItem`-Interfaces hinzugefügt werden, ohne dass der bestehende Code, der mit `MediaItem` arbeitet, geändert werden muss.

3.1.3. Liskov Substitution Principle (LSP)

Beschreibung: Objekte eines Supertyps müssen durch Objekte eines Subtyps ersetzt werden können, ohne die Korrektheit des Programms zu beeinträchtigen.

Beispiel: Die `Book` und `Movie` Klassen sind Subtypen von `MediaItem`.

Begründung: Überall dort, wo ein `MediaItem` erwartet wird, kann auch ein `Book` oder `Movie` verwendet werden, da sie die gleiche Schnittstelle implementieren.

3.1.4. Interface Segregation Principle (ISP)

Beschreibung: Kein Client sollte gezwungen sein, von Methoden abzuhängen, die er nicht verwendet.

Beispiel: Die verschiedenen Repositories (`AccountRepository`, `BookRepository`, etc.) bieten spezifische Schnittstellen für die jeweiligen Aggregate.

Begründung: Anstatt eines generischen Repository-Interfaces gibt es für jedes Aggregat ein eigenes Repository-Interface. Dadurch wird sichergestellt, dass die Clients nur die Methoden sehen, die sie benötigen.

3.1.5. Dependency Inversion Principle (DIP)

Beschreibung: High-Level-Module sollten nicht von Low-Level-Modulen abhängen. Beide sollten von Abstraktionen abhängen.

Beispiel: Die `CreateAccountUseCase` (`org.alcey.bookshelf_pro_application.account.CreateAccountUseCase`) hängt vom `AccountRepository`-Interface ab, nicht von einer konkreten Implementierung.

Begründung: Die konkrete Implementierung des `AccountRepository` (`JooqAccountRepository`) wird zur Laufzeit per Dependency Injection bereitgestellt. Dies entkoppelt die Anwendungslogik von der Datenbankimplementierung.

3.2. Aufgabe 4: Unit Tests

3.2.1. Unit Tests und ATRIP-Regeln

Im Projekt gibt es mehr als 10 Unit-Tests. Die Tests folgen den ATRIP-Regeln (Automatic, Thorough, Repeatable, Independent, Professional).

Beispiel: Der `ReviewUseCaseTest` (`org.alcey.bookshelf_pro_application.ReviewUseCaseTest`) testet die `AddReviewUseCase`.

```
@Test
void testAddReview() {
    // ...
}
```

Begründung: Die Tests sind automatisiert (laufen mit Maven), gründlich (testen verschiedene Szenarien), wiederholbar (produzieren immer das gleiche Ergebnis), unabhängig (können in beliebiger Reihenfolge ausgeführt werden) und professional (sind gut strukturiert und lesbar).

3.2.2. Einsatz von Mocks

In den Unit-Tests werden Mocks verwendet, um Abhängigkeiten zu isolieren.

Beispiel: Im `ReviewUseCaseTest` werden die `ReviewRepository`, `BookshelfEntryRepository` und `CurrentUserProvider` gemockt.

```
@ExtendWith(MockitoExtension.class)
class ReviewUseCaseTest {
    @Mock
    private ReviewRepository reviewRepository;
    @Mock
    private BookshelfEntryRepository bookshelfEntryRepository;
    @Mock
    private CurrentUserProvider currentUserProvider;
    // ...
}
```

Begründung: Durch das Mocking der Repositories und des `CurrentUserProvider` kann der `AddReviewUseCase` isoliert getestet werden, ohne dass eine echte Datenbank oder ein Security-Framework erforderlich ist.

4. Refactoring

4.1. Code Smells

4.1.1. Inconsistent Exception Handling

Identifikation: Vor dem Refactoring im Commit e58ebb31 wurde das Exception-Handling im Projekt inkonsistent gehandhabt.

Begründung: Es gab keine klare Trennung zwischen verschiedenen Arten von Exceptions. `IllegalArgumentException` und `IllegalStateException` wurden teilweise synonym verwendet, was die Fehlerbehandlung erschwerte. Das Refactoring hat eine klare Trennung eingeführt: `SecurityException` für Sicherheitsprobleme, `IllegalArgumentException` für ungültige Benutzereingaben und `IllegalStateException` für interne Fehler.

4.1.2. Duplicate Code

Identifikation: In den Datenbank-Repositories gab es Code-Duplizierung bei der Aktualisierung von Listen-basierten Entitäten (z.B. Tags oder Labels eines MediaItems).

Begründung: Duplizierter Code führt zu einem höheren Wartungsaufwand, da Änderungen (z.B. Fehlerbehebungen) an mehreren Stellen gleichzeitig durchgeführt werden müssen, was fehleranfällig ist. Im Commit 8042003 wurde dieser Smell behoben, indem die gemeinsame Logik in eine generische Hilfsmethode `updateCollection` im `JooqMediaItemRepository` extrahiert wurde (Refactoring: **Extract Method**).

4.1.3. Large Class

Identifikation: In frühen Entwicklungsphasen drohte die `MediaItem`-Entität zu einer **Large Class** (oder **God Class**) zu werden, da sie nicht nur die Kerndaten des Mediums, sondern auch alle dazugehörigen Reviews (Bewertungen) direkt als Liste hielt.

Begründung: Eine Klasse, die zu viele Verantwortlichkeiten bündelt, verletzt das Single Responsibility Principle, ist schwer zu überblicken und erschwert paralleles Arbeiten. Im Commit 749cea0 wurde dieses Problem durch das Refactoring **Extract Class** (im DDD-Kontext: Extraktion in ein eigenes Aggregate Root) gelöst. `Review` wurde zu einem eigenständigen Aggregate, wodurch die `MediaItem`-Klasse deutlich verschlankt wurde.

4.1.4. Speculative Generality

Identifikation: In mehreren Repository-Methoden wurde unnötige Arbeit verrichtet, obwohl die Eingabedaten (z.B. eine leere Liste) keine Aktion erforderten.

Begründung: Code wurde ausgeführt, um z.B. Datenbank-Queries für das Einfügen von Daten vorzubereiten, selbst wenn keine Daten zum Einfügen vorhanden waren. Dies ist ein Beispiel für „Speculative Generality“, bei dem Code für zukünftige oder generische Fälle geschrieben wird, die aktuell nicht eintreten. Im Commit 6a258b0 wurde dies durch das Hinzufügen von „Guard

Clauses“ (die „Early Returns“) behoben. Diese Prüfungen am Anfang der Methoden stellen sicher, dass die Methode sofort verlassen wird, wenn keine Arbeit notwendig ist.

4.1.5. Switch Statements (Type Code)

Identifikation: In der Application Layer (z.B. `GetAccountUseCase`, `GetBookByIdUseCase`) wurden Switch- und If-Statements verwendet, die auf einem expliziten Typ-Feld (`MediaItemType`) der `MediaItem`-Klasse basierten.

Begründung: Die Verwendung von expliziten Typ-Codes und darauf basierenden Fallunterscheidungen ist objektorientiert oft unschön, da bei jedem Hinzufügen eines neuen Typs (z.B. einer Serie) mehrere Switch-Statements in der gesamten Codebase aktualisiert werden müssen, was das Open/Closed Principle verletzt. Im Commit `17fc2ad` wurde dieser Code Smell durch das Refactoring **Replace Conditional with Polymorphism** (und **Replace Type Code with Subclasses**) behoben. Das Feld `type` wurde entfernt, und stattdessen wird Polymorphismus bzw. das Java `instanceof` Pattern Matching eingesetzt, um typspezifische Logik auszuführen.

4.2. Refactorings

4.2.1. Extract Method

Identifikation: Die `execute`-Methode in `UpdateBookUseCase` kann in mehrere kleinere Methoden aufgeteilt werden.

Begründung: Die Validierungslogik, die Berechtigungsprüfung und die Aktualisierungslogik können in separate, private Methoden extrahiert werden. Dies würde die Lesbarkeit und Wartbarkeit der `execute`-Methode erheblich verbessern und die Verantwortlichkeiten klarer trennen. Da dies noch nicht umgesetzt wurde, gibt es keine Commit-ID.

4.2.2. Replace Conditional with Polymorphism / Replace Type Code

Identifikation: Die Ersetzung von auf Typ-Codes basierenden Bedingungsstrukturen.

Begründung: Im Commit `17fc2ad` wurde die Verwendung von `MediaItemType` und dazugehörigen Switch-Statements in Use Cases durch die Nutzung von Polymorphismus (bzw. modernem Java Pattern Matching mit `instanceof`) ersetzt. Dies macht den Code flexibler für zukünftige Erweiterungen von `MediaItem`-Subtypen.

5. Entwurfsmuster

5.1. Einsatz eines Entwurfsmusters: Builder

Im Projekt wird das Builder-Pattern verwendet, um komplexe Objekte zu erstellen.

Beispiel: Der `BookBuilder` (`org.alcey.bookshelf_pro_domain.media_item.book.BookBuilder`) wird verwendet, um ein `Book`-Objekt zu erstellen.

```
Book book = Book.builder(id, owner, title, isbn, pageCount)
    .author(new Author("Jane Doe"))
    .publisher(new Publisher("Awesome Books"))
    .publishDate(new PublishDate(LocalDate.now()))
    .build();
```

Begründung: Das Book-Objekt hat einige Pflichtparameter (wie ID, Titel, etc.), aber auch viele optionale Parameter (Autor, Verlag, Erscheinungsdatum, Cover-Bild). Der Builder vereinfacht die Erstellung des Objekts, indem er eine flüssige API bereitstellt und die Notwendigkeit eines Konstruktors mit zu vielen Parametern vermeidet. Dies verbessert die Lesbarkeit und verringert die Fehleranfälligkeit.